Document number	P3739R0
Date	2025-06-10
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Library Working Group (LWG)

Standard Library Hardening - using std::optional<T&>

Table of contents

Abstract

Utilize `std::optional<T&>` [1] in `inplace_vector` [2] in order to harden it against null pointer dereference errors for similar memory safety reasons as `Standard Library Hardening` [3] and `Minor additions to C++26 standard library hardening` [4].

This was discussed during the standardization of `inplace_vector` [2:1] but could not be adopted since `std::optional<T&>` [1:1] was not sufficiently along in the `C++26` standardization process. Now that `std::optional<T&>` [1:2] is being considered during the June 2025 Sophia meeting, it would be ideal that `inplace_vector` [2:2] underwent minor revisions to better align it with the rest of `C++26` and consequently provide a more optimum interface.

What is being asked for is that 3 of `inplace_vector's` [2:3] modifiers be revised to return an `std::optional<T&>` [1:3] instead of a pointer.

```
template< class... Args >
constexpr ~~pointer~~optional<T&> try_emplace_back( Args&&... args );

constexpr ~~pointer~~optional<T&> try_push_back( const T& value );

constexpr ~~pointer~~optional<T&> try_push_back( T&& value );
```

Motivation

Returning a pointer from these three methods are less than an ideal interface. The primary problem is that it leads to null pointer dereference errors. It is essentially a 100% unsafe interface. Now contrast that with the different iterations of `optional`.

While `optional<T>` can't be used as an alternative to pointer, in `C++17` when it was standardized, `optional` had an equal number of unsafe and safe modifiers. The unsafe ones were `operator->` and `operator*` and the safe modifiers were `value` and `value_or`. In other words, `optional` was born with 50% null pointer dereference safety.

In `C++23`, the `and_then`, `transform` and `or_else` modifiers were added. This brought the total of modifiers to 5 of 7. The result is that 71% of all modifiers are safe from null pointer dereference errors.

However, only in `C++26` with the advent of `std::optional<T&>` [1:4] does `optional` becomes a viable substitute for a nullable pointer. Also in this release is `std::optional range support` [5] which turns `optional` is a range of 0 or 1. This opens the door for 125 range algorithms and 34 range adapters to become null pointer deference safe modifiers for `optional`. Those range algorithms and adapters are growing at a much faster rate than new modifiers being added to the `optional` class itself. This brings the modifer safety rating up to 98%. The original 2 unsafe modifiers and hence the remaining 2% are being addressed by `Standard Library Hardening` [3:1].

Besides the safety benefit, the monadic and range usage are more ergonometric than the legacy pointer based interface.

Wording

23.3.16 Class template inplace_vector [inplace.vector]

23.3.16.1 Overview [inplace.vector.overview]

...

(5.3.3) — If

...

```
  namespace std {
  template<class T, size_t N>
  class inplace_vector {
  public:
```

...

template< class... Args >
constexpr ~~pointer~~optional<T&> try_emplace_back(Args&&... args);

constexpr ~~pointer~~optional<T&> try_push_back(const T& value);

constexpr ~~pointer~~optional<T&> try_push_back(T&& value);

...

```
  };
  }
```

...

23.3.16.5 Modifiers [inplace.vector.modifiers]

...

template<class... Args>
constexpr ~~pointer~~optional<T&> try_emplace_back(Args&&... args);
constexpr ~~pointer~~optional<T&> try_push_back(const T& x);
constexpr ~~pointer~~optional<T&> try_push_back(T&& x);

[8] Let `vals` denote a pack:

(8.1) — `std::forward<Args>(args)...` for the first overload,

(8.2) — `x` for the second overload,

(8.3) — `std::move(x)` for the third overload.

[9] *Preconditions*: `value_type` is `Cpp17EmplaceConstructible` into `inplace_vector` from `vals...`.

[10] *Effects*: If `size() < capacity()` is `true`, appends an object of type `T` direct-non-list-initialized with `vals...`. Otherwise, there are no effects.

[11] *Returns*: `nullptr` if `size() == capacity()` is `true`, otherwise `addressof(back())`.

[12] *Throws*: Nothing unless an exception is thrown by the initialization of the inserted element.

[13] *Complexity*: Constant.

[14] *Remarks*: If an exception is thrown, there are no effects on `*this`.

Impact on the standard

Other than `inplace_vector` [2:4], there are no other changes to the library standard and since `C++26` has not be released, this tweak is not expected to cause any problems.

The proposed changes are relative to the current working draft `N5008` [6].

References

1. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2988r12.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2988r12.pdf) ↩ ↩ ↩ ↩ ↩
2. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p0843r14.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p0843r14.html) ↩ ↩ ↩ ↩ ↩
3. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3471r4.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3471r4.html) ↩ ↩
4. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3697r0.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3697r0.html) ↩
5. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3168r2.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3168r2.html) ↩
6. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf) ↩